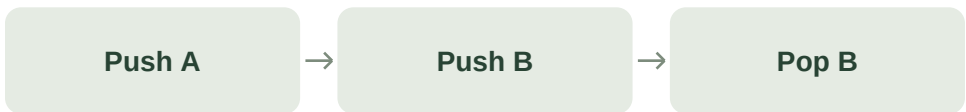


Computer Science Essentials

A model is a simplification

A useful model keeps the details that affect a decision and leaves out those that do not. It should help you reason, not merely give something a more impressive name.

For example, describing a stack as last-in, first-out helps predict which item is removed next. The description is useful before deciding how to store the stack.



Use a tiny concrete example before introducing an abstraction.

Growth, not a stopwatch

Time complexity describes how work grows with input size under a stated model. It does not predict the exact time on a particular device.

A linear scan may inspect every element. A binary search repeatedly narrows a sorted search space. The latter relies on a precondition: the data must have a compatible ordering.

Ask whether preparation and sorting costs are included in the analysis.

References and copies

Two variables can refer to the same mutable object. Changing that object through one variable is visible through the other reference.

```
const a = { score: 1 };  
const b = a;  
b.score = 2;  
console.log(a.score); // 2
```

Copying an outer object with spread does not automatically copy its nested objects. The boundary between shared and copied data matters.

Draw two arrows pointing to the same object. The bug becomes easier to see.

Use the right structure

The important question is not which data structure is most sophisticated. It is which operations must be cheap, which ordering matters and what guarantees the caller needs.

A library may keep its original files by stable identifier while maintaining a separate searchable metadata representation. Different representations serve different operations.

Original sample material. JavaScript reference semantics: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects

Before optimizing, list the three operations your application performs most often.